EGC 211P Programming 2A (C++) GIE

Total Duration: 2 hours

Maximum Marks: 100

Download main.cpp from the contest, add code to it, and submit.

FILES TO SUBMIT: main.cpp DO NOT DELETE ANYTHING FROM main.cpp

If you delete any existing code, your code may get compilation error.

*There are 6 classes to be designed (Q1-Q6), marks for each are specified along with the question. Q7 is to write **main function,** with dedicated marks. Unless Q1-Q6 are completed, you will get compilation error, so do not worry about it at the beginning of coding. Once Q1-Q6 are done, verify compilation, and then complete the main function. Then only DomJudge testcases will run.*

STL help document: http://172.16.201.202/cpp/stl.pdf

Problem Context:

*You are given the task of implementing a **JOIN operator** that joins two tables of Tuple objects, where each table is stored as a vector<Tuple>. The join operation is performed based on a **common key column** (key). The output of the join operation is a vector<Tuple>. The JoinOperator class has **two concrete implementations**: NestedLoopJoin, and HashJoin. Each implementation must produce **identical logical results** for the same input tables -- rows would be the same, but the sequence may be different. Here is one example to show the working of our JOIN.*

left TABLE (stored as `vector<Tuple> left`)

Key	Value
1	10
2	20
3	30
2	25

right TABLE (stored as `vector<Tuple> right`)

Key	Value
2	200
3	300
4	400
1	250

*The content of the output table is calculated as: output = (key, l.value + r.value) for **every matching pair** (l, r). That is, when l.key == r.key, exactly one output tuple is produced. So, if a key is present only in one of the tables but not in the other, it does not appear in the output. Duplicate keys are ok – if*

*there are **m** Tuples in left with **key = k**, and n Tuples in right with **key = k**, total **m*n** Tuples will be in the output table with **key = k**.*

OUTPUT TABLE (can be used to output a vector<Tuple> result)

Key	Value
1	260
2	220
3	330
2	225

Q1. Define the following two classes exactly as specified. ***10 Marks***

1. Class Name: **Tuple**
   Private Data Members
   int **key**
   int **value**

2. Class Name: **HashEntry**
   Private Data Members
   int **key**
   int **value**

3. Constructor Requirements
   Each class must define a **proper parameterized constructors** with the following signature:
   Tuple(int key, int value)
   HashEntry(int key, int value)
   The constructor must initialize the corresponding private data members so that objects such as the following can be created:
   Tuple(2, 225) // key = 2, value = 225
   HashEntry(3, 35) // key = 3, value = 35

4. **Getter** Methods:
   Each class must define proper getter methods to enable access to its private data members.

Q2. Define JoinOperator class exactly as specified. **10 Marks**

Class Name: **JoinOperator**

Protected Members

1. vector<Tuple> left: Stores the left input table.
2. vector<Tuple> right: Stores the right input table.

3. vector<Tuple> result: Stores the output of the join operation. Initially empty.

Public Members

1. Constructor: JoinOperator(vector<Tuple>& left, vector<Tuple>& right)
   -- Initializes the data members left and right by the input vectors.
2. Destructor:
   a. Must be declared virtual (derived classes must have proper overrides)
3. *Pure* Virtual Method (to make JoinOperator class an abstract): **void execute()**
   a. Performs the join operation (in derived classes).
   b. Populates the result vector (in derived classes).
4. Getter Method: **vector<Tuple> getResult() const**
   a. Returns the result vector by value.

Q3. Make a derived class NestedLoopJoin from JoinOperator as follows.

15 Marks

Class Name: **NestedLoopJoin**

Public Members

1. Constructor: Must take the left and right vectors as input.
   Must invoke the base class (JoinOperator) constructor to initialize the inherited left and right data members.
2. Method: **execute**:
   Join Logic:

```
FOR each Tuple l in left
    FOR each Tuple r in right
        IF l.getKey() == r.getKey() THEN
            Add a Tuple to result with:
            - key = l.getKey()
            - value = l.getValue() + r.getValue()
```

Q4. Define class HashTable as specified below. 20 Marks

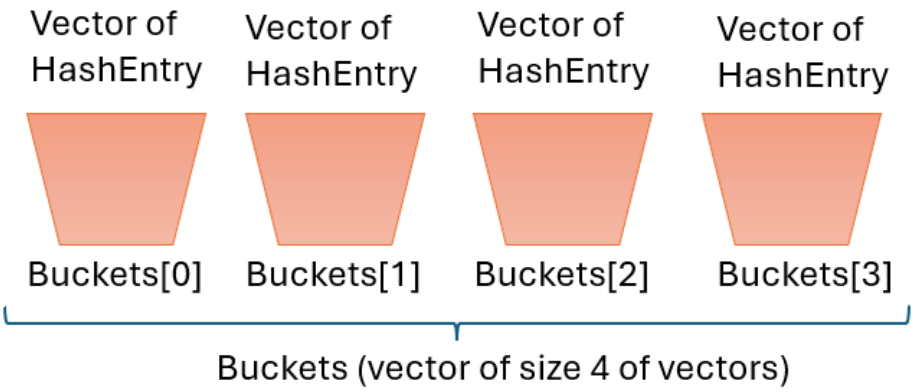
Class Name: HashTable

Private Members

1. vector<vector<HashEntry>> buckets

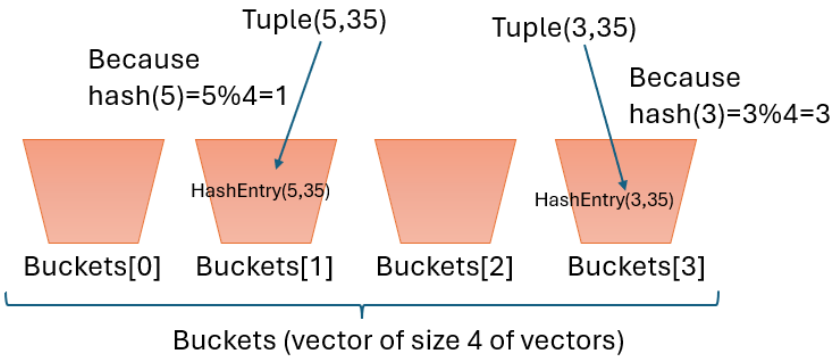
   buckets represents a hash table. Each index in buckets stores a bucket, which is a vector of HashEntry objects. HashTable class keeps all its data in the form of

HashEntries, in buckets. Here is one picture to visualize the interpretation of this member:



2. Method signature: **int hash(int key)**
   -- Method to compute the hash index for a given key.
   Behaviour:
   IF buckets.size() == 0,
   Throw runtime_error("No entry")
   ELSE,
   Return key % size of buckets

*Example: The purpose of the method **hash** is to put Tuples into buckets as HashEntries, as shown in the picture below. The **insert** method of this class, as specified later among the public members, puts the HashEntries to the hash buckets by computing hash function.*



Public Members

1. Constructor: HashTable(int size)

   -- Initializes buckets as a **fixed-size vector of length size** using *initializer list* – *hint: call buckets(size) in the initializer list.*

   -- Each bucket is initially an empty vector of HashEntry.

1. Method signature: **void insert(int key, int value)**
   Behaviour: Refer to the picture before.
   a. Create a HashEntry object **h** using the given key and value.

b. Compute the hash index **idx** by calling the **hash(key)** method of this class.
c. Insert **h** into buckets at the index **idx**.

2. Method signature: **vector<HashEntry>& getBucket(int key)**
   Behaviour:
   a. Compute the hash index **idx** by calling the **hash(key)** method of this class.
   b. Return a reference to the bucket at index **idx**.
   Example: In the above picture, for key 3, hash(3) = 3%4 = 3. So getBucket(3) should return the buckets[3] vector object by reference.
   Exception Behaviour:
   c. If bucket is not found at index **idx**, throw runtime_error("No entry")

Q5. Make a derived class HashJoin from JoinOperator as follows: **25 marks**

It is one JOIN implementation that uses HashTable.

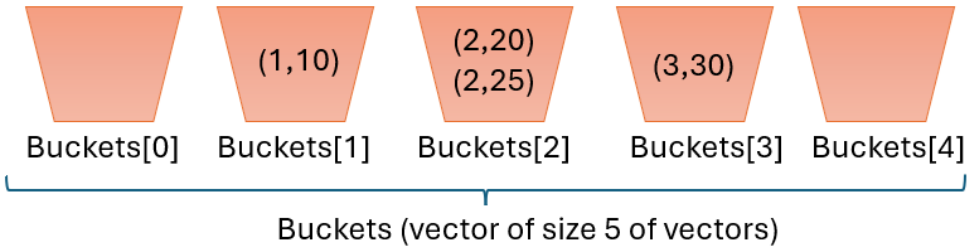
Class Name: HashJoin

Private Members

1. **HashTable* ht**: Pointer to a dynamically allocated HashTable object.
2. Method signature: **void populateHashTable()**
   Behaviour:
   For each Tuple t in left (run for loop on left), call the insert method of the HashTable class through ht. The parameters to the insert method should be: key = t.getKey(), value = t.getValue(). This will add a corresponding HashEntry in the hashTable.

Example: For the following left table, here is the HashTable.

Key	Value
1	10
2	20
3	30
2	25



3. Method signature: **void FindMatch(Tuple& t, vector<HashEntry>& bucket)**
   Behaviour:
   For each HashEntry h in the parameter bucket
   IF h.getKey() == t.getKey(), then:
   Add a Tuple to result with:
   key = t.getKey(), value = h.getValue() + t.getValue()

Example: For a Tuple t=(2,200), let us say FindMatch is called with buckets[2] reference object. Here key =2. So, for both the entries (2,20) and (2,25) in buckets[2], t will find match, and the result vector will be added with 2 tuples: (2, 220), and (2, 225).

Public Members

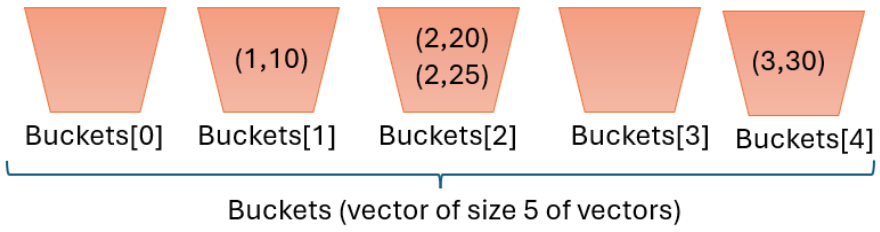
1. Constructor: Must take the left and right vectors as input.
   Behaviour:
   a. Must invoke the base class (JoinOperator) constructor to initialize the inherited left and right members.
   b. Must dynamically allocate a HashTable (of size left.size() + 1) object using the **new** operator, and ht should point to the HashTable.

   In the above image, left vector was having 4 tuples, so HashTable has 5 buckets.

2. Destructor: properly deallocate the hash table using **delete** operator.
3. Method: **execute**()
   Join Logic:
   First, Calls **populateHashTable**() to build the hash table using the left vector.
   Then, For each Tuple t in the inherited right vector (for loop)
   Bucket = ht->**getBucket**(t.getKey())
   Calls **FindMatch**(t, bucket) to find matching keys and populate result.

For the following right table, here is the sequence of actions happening to perform the join.

*Output of **populateHashTable()** is the following*



Given the following right table, here is the sequence of Tuple going through the for loop.

Key	Value
2	200
3	300
4	400
1	250

(2,200), (3, 300), (4, 400), (1, 250)

Let's say current tuple is (2,200). So, getBucket returns index hash(2)=2%5 = 2.

Since HashEntry (2,20) is present in buckets[2], and they have the same key, output tuple (2,220) is added to result.

Since HashEntry (2,25) is present in buckets[2], and they have the same key, output tuple (2,225) is added to result.

Let's say current tuple is (4,400). So, getBucket returns index hash(5)=4%5 = 4.

Since there is no HashEntry in buckets[4], no output tuple gets added in result.

Let's say current tuple is (1,250). So, getBucket returns index hash(1) = 1%5 = 1.

Since HashEntry (1,10) is present in buckets[1], and they have the same key, output tuple (1,260) is added to result.

Q6. 20 Marks: Fill-in the skeleton code of main() to take input from the user for joining. If your JOIN implementations are correct, correct output will be automatically produced by the code, for any input you give.

Test with the inputs from the Domjudge testcases.